설비 중단과 대체 배정 판단

> 지금 연구에서 온톨로지와 SHACL의 가장 설득력 있는 역할은 LLM의 지능을 높이는 것보다 여러 구성요소가 같은 의미로 데이터를 읽고 판단하도록 만드는 것입니다.

"설비 M02가 멈췄습니다. 주문 O17과 O18을 M03으로 돌려도 되나요?"

현장 배정 담당자

M02가 멈췄는데 O17·O18을
설비 M03으로 돌려도 될까?

→

품질·자격 담당자

갱신 기록 Q03R이 두 제품의
역량·유효기간에 모두 맞을까?

→

감사·운영 담당자

어떤 기록과 규칙 버전으로
판단했고 무엇을 재검토할까?

발표 구성

01–05 · 업무 의미와 규칙

대체 가능의 네 층위 구분
공통 규칙 공유와 적재·통합 점검

06–21 · 추론과 SHACL 실습

RDF · RDFS · OWL · SHACL 구분
제조 부품 예제와 노트북 실행

22–25 · 도구·기록·도입 기준

LLM 도구 계약 · 검토 근거 보존
저장소 대조(48쌍)와 도입 질문

- **연구 방향:** 온톨로지·SHACL의 역할 구분과 여섯 가지 실무 가치
- **의미와 추론:** 제조 부품 예제로 보는 RDF·RDFS·OWL과 추론의 경계
- **SHACL 검증:** 조건 명세, 노트북 실습, 저장소·도구·검토 기록 연결
- **도입 기준:** 공유 Python 대안과의 비교, 도입 판단 질문

데이터 구조에 업무 의미 남기기

> 공장 사례에서 `M03 → Q03R` 이라는 연결만으로는 부족합니다. 기능적 가능, 유효 자격, 일정 여유, 사용 승인을 구분해 명시해야 개발자와 LLM이 모두 '대체 가능'으로 뭉뚱그리지 않습니다.

'대체 가능'으로 뭉뚱그리기 쉬운 네 가지 업무 의미 구분

가상 공장 실험 사례 (M03 · Q03R · O17/O18)

① 기능적 작업 가능

M03 → CAP-CUT

절단 역량이 등록된 설비
O17·O18 모두 기능 후보

자격·일정은 미확인

② 유효한 제품 자격

M03 → Q03R (P-A)

Q03R은 제품 P-A만 유효
O18(제품 P-B)은 범위 불일치

제품·역량·기간 동시 충족 필요

③ 일정 배정 가능

available ≠ 일정 여유

설비 상태값만으로는
실제 부하·납기 판단 불가

별도 스케줄 확인 대상

④ 사용 승인 완료

운영·품질 변경 승인

현장 소문("갱신했다더라")
≠ 공식 승인된 배정 변경

담당자 확인·승인 절차 분리

온톨로지의 역할: 관계를 무조건 늘리는 것이 아니라 위 네 가지 구분을 데이터 구조에 명시하는 것

다만 RDF로 표현한다고 구분이 저절로 생기지는 않으므로, 개념과 적용 조건을 직접 설계해야 한다.

- 자격 범위: `Q03R` 이 어떤 제품(`P-A` / `P-B`)·역량(`CAP-CUT`)·유효기간에 적용되는지 구분
- 상태 의미: 설비의 `available` 이 생산 일정상 여유까지 뜻하는지 별도 정의

공통 규칙 공유와 공정한 비교 기준

> SHACL을 공통 규칙 산출물로 두면 여러 서비스가 어떤 버전의 조건으로 검사했는지 추적하기 쉬워집니다. 공정한 비교 대상은 공유 스키마·규칙·버전 관리를 갖춘 Python 시스템입니다.



- 챗봇·검색 API·품질 점검·데이터 적재가 각각 조건을 구현하면 한쪽만 수정될 위험
- 차이는 규칙이 특정 애플리케이션 코드에 묶이는 정도와 다른 도구에서 재사용할 수 있는 정도

파이프라인 전반의 품질 관리 지점

> SHACL은 최종 답변 직전뿐 아니라 데이터 적재·통합·정책 변경·질의 전달 단계에서도 유용합니다. 다만 검사 통과가 사실의 진위를 보증하지는 않습니다.

1. 새 데이터 적재

필수 식별자·타입·참조

누락·자료형 오류를
적재 직후 확인

2. 데이터 통합

출처 간 관계 의미 대조

같은 관계명을 다른 뜻으로
쓰지 않는지 점검

3. 자격·정책 변경

기존 기록의 재검토 범위

바뀐 조건을 만족하지 못하는
후보만 표적 추출

4. 질의 결과 전달

판단 근거의 입력 완전성

원천 결함과 조회 누락을
구분해 LLM에 전달

지점	확인할 수 있는 내용
새 데이터 적재	필수 식별자·타입·참조가 갖춰졌는가
데이터 통합	두 출처가 같은 관계를 서로 다른 의미로 쓰고 있지는 않은가
자격·정책 변경	새 규칙에서 어떤 기존 기록이 조건을 만족하지 못하는가
질의 결과 전달	이번 판단에 필요한 필드와 연결이 제공됐는가

추론·검증·사실 확인의 구분

> 입력이 틀리면 논리적으로 일관된 결론도 현실에서는 틀릴 수 있습니다.

추론

정의와 기록으로부터
어떤 결론이 따라오는가

→

데이터 검증

제출한 데이터가
정해 둔 조건을 만족하는가

→

현실의 사실 확인

원천 근거·현장·전문가로
입력과 결론을 확인

작업	묻는 질문	필요한 근거
추론	정의와 기록에서 어떤 결론이 따라오는가	입력 사실 + 공리·규칙
데이터 검증	제출한 데이터가 조건을 만족하는가	검사할 그래프 + Shape
현실의 사실 확인	기록과 결론이 실제 업무에서 맞는가	독립 자료·현장 확인·전문가 판단

RDF·RDFS·OWL·SHACL의 역할

> 온톨로지는 무엇을 어떤 관계로 해석할지 정하고 SHACL은 특정 용도의 조건 충족을 검사합니다. 둘 중 하나만 쓰는 설계도 가능합니다.

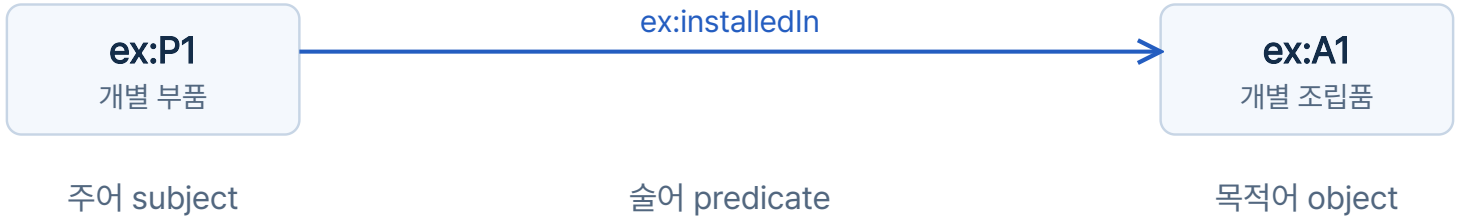
구성	역할	제조·설비 예
RDF	개체·관계를 트리플로 표현	P1은 조립품 A1에 장착된다
RDFS	클래스 포함 관계, domain/range 등 정의	볼트는 체결 부품, 체결 부품은 구성 부품
OWL	더 풍부한 논리적 공리 표현	역관계, 클래스 조합·배타성, 동일성
추론기	지원하는 공리·규칙의 결론 계산	P1도 구성 부품이라는 타입 도출
SHACL	주어진 RDF의 조건 충족 검사	필수 제조사·자격 범위가 기록됐는가

출처: RDF · RDFS · OWL · SHACL

RDF 트리플과 제조 부품

> RDF는 개체와 관계를 식별자로 연결합니다. Turtle은 그 그래프를 텍스트로 적는 형식입니다.

관계 하나를 주어 · 술어 · 목적어로 기록



ex: = https://example.org/manufacturing/

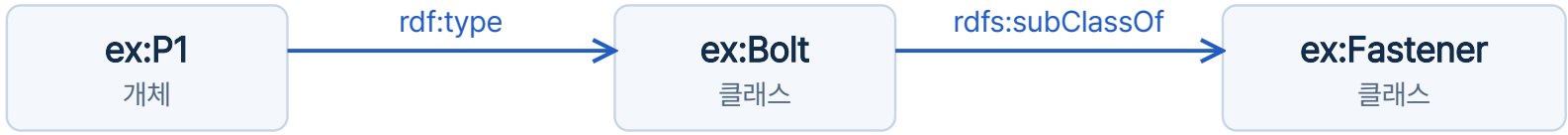
```
@prefix ex: <https://example.org/manufacturing/> .
ex:P1 a ex:Bolt ;                # P1은 볼트 개체
    ex:installedIn ex:A1 .       # P1은 조립품 A1에 장착
```

- 입력은 설명용 합성 부품 기록이며 실제 설치 사실 확인과는 별개

개체·클래스·포함 관계

> P1은 하나의 부품이고 Bolt는 그 부품이 속한 클래스입니다. 클래스 간 포함 관계와 개체의 소속을 구분합니다.

개체의 타입과 클래스의 포함 관계



rdf:type은 개체 → 클래스, rdfs:subClassOf는 클래스 → 클래스 관계다.

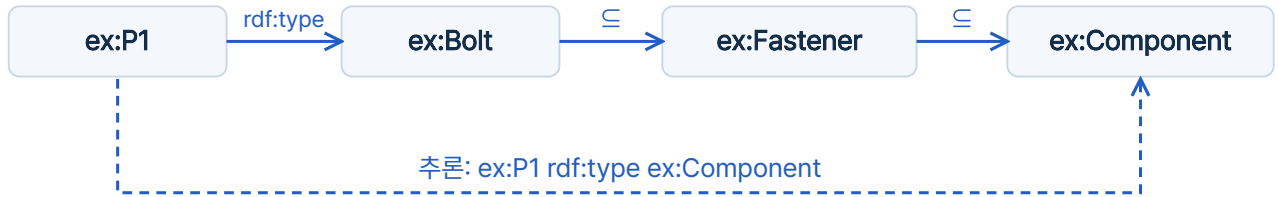
```
ex:P1 a ex:Bolt .                          # 개체 → 클래스
ex:Bolt rdfs:subClassOf ex:Fastener .      # 클래스 → 상위 클래스
```

출처: RDFS 클래스와 인스턴스

RDFS 클래스 계층과 타입 추론

> 하위 클래스의 개체는 상위 클래스에도 속합니다. 점선은 정의를 적용해 얻는 추가 결론입니다.

클래스 포함 관계에서 개체 타입 도출



⊆ = rdfs:subClassOf · 실선은 명시한 관계, 점선은 규칙으로 도출한 관계

- 입력: `P1 a Bolt` + `Bolt ⊆ Fastener ⊆ Component`
- 출력: `P1 a Fastener`, `P1 a Component`
- 실제 부품의 정체가 맞는지는 원천 자료로 별도 확인

출처: RDFS

domain·range의 타입 해석

> domain과 range는 관계의 주어·목적어를 어떤 타입으로 해석할지 정합니다. 입력을 거부하는 검증 규칙과는 다릅니다.

domain · range는 관계 양끝의 타입을 도출



타입 추론 규칙이다. 허용 타입을 검사하는 입력 검증 규칙과 구분한다.

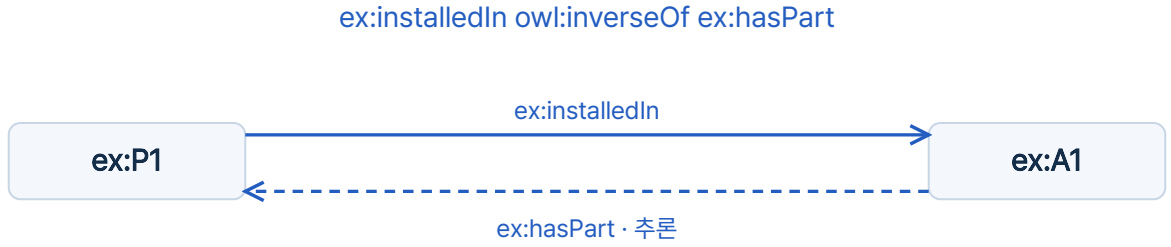
```
ex:installedIn rdfs:domain ex:Component ; # 주어는 구성 부품
               rdfs:range ex:Assembly . # 목적어는 조립품
```

출처: RDFS domain/range

OWL 공리와 역관계

> OWL의 공리는 관계와 클래스의 논리적 의미를 명시합니다. 실제 계산 범위는 선택한 추론기의 지원 기능에 달려 있습니다.

역속성 정의로 반대 방향 관계 도출



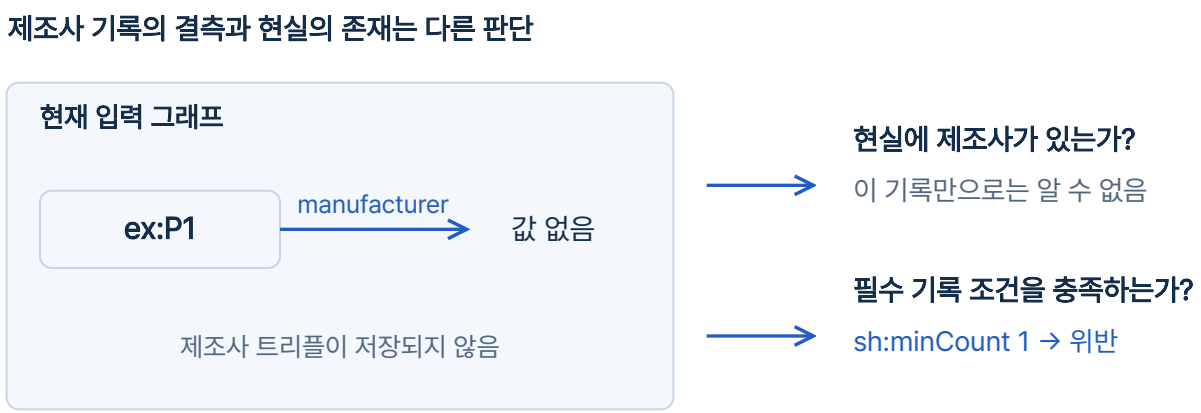
역방향 관계는 owl:inverseOf 선언이 있을 때 도출한다.

- `inverseOf` : P1이 A1에 장착되면 A1은 P1을 포함
- `equivalentClass` : “인증 부품이면서 체결 부품” 같은 분류 정의
- `disjointWith` / `sameAs` : 배타 분류 / 동일 개체 선언

출처: OWL 2 Primer

열린 세계와 기록 부재

> 기록이 없다는 사실만으로 현실에서 거짓이라고 결론 내릴 수 없습니다. 제출 자료의 필수 항목 검사는 별도로 선언합니다.



OWL의 열린 세계 해석과 SHACL의 입력 그래프 검사를 구분한다.

- OWL: 추가 근거가 없으면 제조사나 갱신 자격의 실제 존재 판단을 유보
- SHACL: 제출된 기록에 필수 식별자가 없으면 `minCount` 위반

출처: OWL 열린 세계 가정

전체 조건·존재 조건·동일성

> “모두 인증 부품”이라는 조건은 연결 대상의 존재를 보장하지 않습니다. 식별자 두 개도 서로 다른 실체 둘을 자동으로 뜻하지 않습니다.



점선의 ?는 의미상 존재 요구를 나타낸다. 저장된 트리플이나 생성된 개체가 아니다.

- `allValuesFrom`: 연결된 대상이 있다면 지정 클래스에 속함
- `someValuesFrom`: 지정 클래스에 속한 대상의 존재를 요구
- 다른 개체임을 명시하려면 별도 동일성·차이 선언과 근거 검토

출처: OWL 제약과 동일성

추론으로 추가되는 조회 결과

추론을 쓰는 목적: 직접 입력하지 않은 결론도 정의를 따라 찾아내는 것

기록과 정의

입력 기록

P1은 볼트다.
P1은 조립품 A1에 장착된다.

함께 제공한 정의

볼트 ⊆ 체결 부품 ⊆ 구성 부품
장착된다 ↔ 포함한다 (역관계)

추론 후 추가로 조회할 수 있는 결론

P1은 구성 부품이다.

A1은 P1을 포함한다.

직접 적지 않은 타입과 역방향 관계를
입력과 정의를 결합해 계산한다.

- 다음 장에서는 같은 입력에 RDFS·OWL-RL을 적용한 실제 조회 결과를 비교한다.

원본·RDFS·OWL-RL의 조회 결과

> 같은 입력이라도 적용한 정의와 조회 범위에 따라 보이는 결론이 달라집니다. 제조사 기록은 세 조건 모두 비어 있습니다.

저장된 조회 결과 · 없음은 현실의 거짓을 뜻하지 않음

조회 결과	원본	RDFS	OWL-RL
P1 a Component	없음	있음	있음
A1 a Assembly	없음	있음	있음
A1 hasPart P1	없음	없음	있음
제조사 기록	없음	없음	없음

- 입력: P1의 볼트 타입, A1 장착 관계, 클래스·역관계 정의
- 출력: 부품 타입·조립품 타입·역관계의 존재 여부
- 버튼은 실제 저장 결과를 비교하며 새 Python 실행을 시작하지 않음

다단계 관계와 SKOS·OWL 구분

> 다단계 연결 계산은 도달 가능한 대상 쌍을 구하며 용어 분류(`skos:broader`)와 클래스 포함(`rdfs:subClassOf`)은 서로 다른 관계입니다.

개념 분류와 클래스 분류의 서로 다른 연결



skos:broader와 rdfs:subClassOf는 다른 의미다. broader 자체를 추이적 속성으로 읽지 않는다.

- 다단계 도달: `(P1, A1), (A1, PRODUCT_X)`에서 `(P1, PRODUCT_X)` 도출
- 모델링 역할 구분: SKOS는 용어·분류 체계 연결, OWL/RDFS는 개체 타입과 논리 관계 정의

출처: SKOS Primer · OWL 2 Primer

SHACL의 입력과 검사 결과

> SHACL 통과는 선언한 조건의 충족입니다. 기록된 정보가 현실에서 참이라는 인증은 아닙니다.



SHACL: Shapes Constraint Language. **Shape**: 검사 대상과 조건을 묶은 구조.

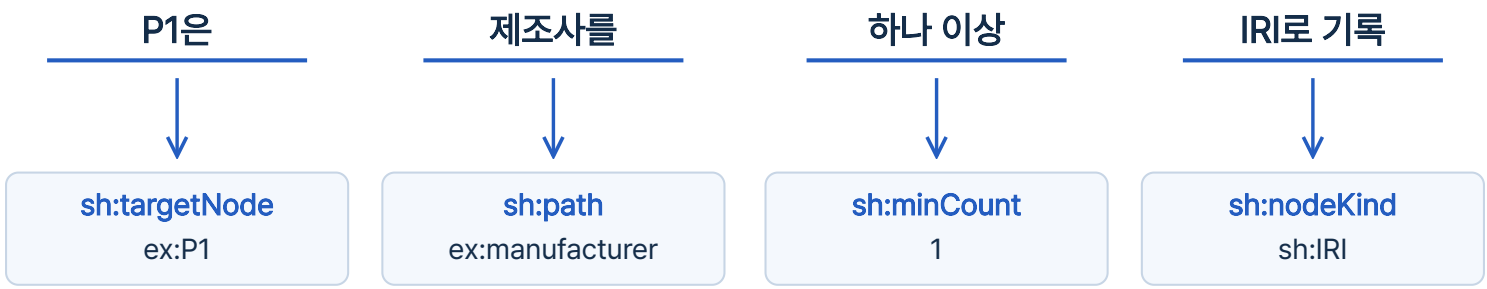
- Data Graph: 검사할 부품·설비 기록
- Shapes Graph: "필수 식별자와 적용 범위 조건이 기록돼야 한다"
- Validation Report: 어떤 개체·속성이 어떤 조건을 어겼는지 구조화해 기록

출처: SHACL

업무 문장의 Shape 변환

> Shape 작성은 요구 문장을 검사 대상·속성·개수·값 종류로 나누는 과정입니다. 뜻이 맞는지는 정상·위반·결측 반례로 확인합니다.

문장의 각 조건을 Shape 속성에 대응



대상 선택: targetNode / targetClass · 개수 조건: minCount / maxCount

Shape는 요구사항을 명시한다. 검사 통과가 제조사의 실제 존재나 정확성을 증명하지는 않는다.

- `targetNode`는 특정 개체, `targetClass`는 클래스의 대상 개체를 선택
- 문법이 맞는 Shape도 잘못된 업무 조건을 검사할 수 있으므로 반례 점검이 필수

노트북 실습 — 데이터와 Shape 준비

- P1의 기록과 제조사 필수 조건을 각각 RDF 그래프로 읽는다.

manufacturing-shacl.ipynb · Python 3 저장된 실행 결과 재생

In [2] data

```
DATA = """ex:P1 a ex:Bolt ;
    ex:installedIn ex:A1 ."""
data = Graph().parse(data=PREFIX + DATA, format="turtle")
print(f"데이터: {len(data)}개 트리플")
```

Out [2]

```
데이터: 2개 트리플
```

In [3] shape

```
SHAPE = """ex:ManufacturerShape a sh:NodeShape ;
    # 이 검사는 P1의 제조사 기록만 확인한다.
    sh:targetNode ex:P1 ;
    sh:property [
        sh:path ex:manufacturer ;
        # 값이 하나 이상이고 IRI인지 검사한다.
        sh:minCount 1 ;
        sh:nodeKind sh:IRI ;
        sh:message "제조사 식별자를 하나 이상 기록하세요."
    ] ."""
shapes = Graph().parse(data=PREFIX + SHAPE, format="turtle")
print(f"Shape: {len(shapes)}개 트리플")
```

Out [3]

```
Shape: 7개 트리플
```

▶ 셀의 실제 저장 출력 표시

노트북 실습 — 검사와 보고서 확인

- `conforms`는 조건 충족 여부, `report`는 위반 근거, `report_text`는 텍스트 보고서다.

manufacturing-shacl.ipynb · Python 3 저장된 실행 결과 재생

In [4] validate

```
conforms, report, report_text = validate(
    data, shacl_graph=shapes, inference="none")
print("conforms =", bool(conforms))
```

Out [4]

```
conforms = False
```

In [5] report

```
for result in report.subjects(RDF.type, SH.ValidationResult):
    print("대상:", report.value(result,
SH.focusNode).n3(data.namespace_manager))
    print("속성:", report.value(result,
SH.resultPath).n3(data.namespace_manager))
    print("조건:", str(report.value(result,
SH.sourceConstraintComponent)).split("#")[-1])
    print("메시지:", report.value(result, SH.resultMessage))
```

Out [5]

```
대상: ex:P1
속성: ex:manufacturer
조건: MinCountConstraintComponent
메시지: 제조사 식별자를 하나 이상 기록하세요.
```

▶ 셀의 실제 저장 출력 표시

LLM 도구 설계의 명시적 계약

> 미리 만든 쿼리를 LLM이 선택하는 구조에서는 도구마다 반환 범위와 후속 확인 조건을 계약으로 선언해 프롬프트 속 숨은 가정을 시스템 구성요소로 옮길 수 있습니다.

도구 계약 선언 · `replacement_candidates(order_id)`

> "이 도구는 요구 역량(CAP-CUT)을 갖춘 **기능적 후보 설비**만 반환한다. 자격이나 일정 여유까지 확인하지 않으므로 다음 판단에는 별도 자격 조회(`qualification_records`)가 필요하다."

도구 이름	반환하는 근거	단독으로 확정할 수 없는 것
`replacement_candidates`	요구 역량을 갖춘 설비 후보	제품별 유효 자격 · 실제 일정 여유
`qualification_records`	설비의 자격 코드·제품·유효기간	작업장 부하 · 납기 준수 · 현장 승인
`impacted_orders`	특정 설비에 배정된 주문 목록	대체 설비의 적격 여부

온톨로지 · SHACL · 검사기의 역할 분담

> **온톨로지 (개념 정렬):** 도구가 반환하는 `Machine`, `Capability`, `Qualification`의 관계와 상태 정의를 통일

> **SHACL (입력 근거 검사):** 최종 판단 전에 선택된 후보에 대해 제품·역량·기준일 자격 근거가 실제로 조회됐는지 확인

> **명시적 한계:** LLM이 질문을 잘못 해석했거나 엉뚱한 도구를 선택한 문제까지 검사기가 자동으로 해결하지는 못함

출처: `apps/factory_lab/graph.py` · `docs/journal/2026-11-10-factory-query-tools-and-branches.md`

- 온톨로지는 개념을 정렬하고 SHACL은 입력 조건 충족을 검사하되 도구 오선택·질문 오해석까지 해결하지는 못함

사람이 검토할 구조화된 근거 기록

> 최종 답변이 “M03은 안 됩니다”로 끝나는 것과 판단 대상·적용 조건·확인 기록·구체적 사유·규칙 버전을 구조화해 남기는 것은 다릅니다.

단순 자연어 거절 vs 구조화된 검토 기록

> **단순 답변:** “M03은 안 됩니다.” → 왜 안 되는지, 어떤 기록을 봤는지, 규정 변경 시 다시 봐야 하는지 알 수 없음

구조화된 검토 기록 (O18 · M03 사례) — 별도 보존

판단 대상	주문 `O18` (`P-B`)에 대한 대체 후보 `M03`
적용 조건	단일 자격 내 제품(`P-B`)·역량(`CAP-CUT`)·유효기간 충족
확인한 기록	`Q03` (만료), `Q03R` (`P-A` 전용 · 2026-11-09 기준 유효)
구체적 문제	자격 기록은 있으나 제품 범위 불일치 (`qualification-not-applicable`)
데이터·규칙 버전	데이터 스냅샷 `factory-v3` · 자격 정책 계약 `v1`

이번 실험에서 드러난 보존 필요성

가상 공장 갱신 실험(`factory-renewal-live`)에서 LLM 최종 자연어 요약은 후보별 설명과 요약표에서 **자격 범위 불일치(`not-applicable`)**와 **자격 미조회 (`not-retrieved`)**를 섞어 표현했다.

활용 장면	구조화된 기록이 하는 역할
이의 제기 · 감사	어떤 근거 ID(`Q03` , `Q03R`)로 배정을 보류했는지 즉시 대조
오류 재현 · 디버깅	조회 누락인지 자격 불일치인지 자연어 환각 없이 분리
담당자 인계	품질팀에 `P-B` 추가 인증 여부를 물을지 명확히 전달
정책·데이터 변경	새 자격 `Q03R` 추가 시 `O17` 만 해소되고 `O18` 은 유지됨을 추적

출처: `docs/experiments/records/2026-11-10-factory-renewal-live.json`

- 최종 자연어가 사유(`not-applicable` vs `not-retrieved`)를 혼용해도 구조화 기록을 별도 보존해 감사·오류 재현·인계·재검토에 활용

저장소 변경 시 의미를 유지하는 기준

> Neo4j와 RDF 저장소는 표현과 쿼리가 달라도 "같은 후보·같은 근거·같은 조건"을 반환해야 비교가 가능합니다. 공통 의미 모델은 이때 기준점이 됩니다.

저장소·표현이 달라도 같은 기준 유지

Neo4j(Cypher)와 RDF 저장소(SPARQL)는 내부 구조와 질의어가 다르므로 **같은 후보·근거·조건**을 반환하는 공통 의미 모델이 비교 기준이 된다.

실험 범위	비교 대상	관측 결과
가상 공장 도구 대조	Neo4j Cypher vs RDFLib SPARQL (3버전 × 16호출)	**48쌍 조회 100% 일치** · 6개 묶음 플래그 일치
기본 그래프 패턴	SQL/Python · RDFLib · Neo4j · Neptune	실행된 36개 슬롯 모두 고정 참조값과 일치
계층 추론·삭제·복원	Python · 외부 RDFS · Neo4j+n10s · Neptune	갱신 조건 32/32 일치 · 미갱신 불일치 4건 포착

변경 후 재검토 범위를 좁히는 질문 계약

데이터·규칙 변경 후 전체 묶음 해시만 비교하면 답이 같은 결과에도 경고가 붙는다. 선언한 질문 대상·연결 경로 해시를 쓰면 불일치 8건은 유지한 채 경고를 줄인다.

확인 범위 정책 (160개 결과 재검사)	전체 경고	실제 불일치 포착	일치 결과에 붙은 경고
전체 묶음 해시 비교	32	8 / 8	24
선언한 질문 유형별 범위	20	8 / 8	12
선언한 조회 대상·연결 경로	**10**	**8 / 8**	**2**

작은 증거: 기준 구현·확인 가능성 입증 · 가상 공장 48쌍의 Neptune 실증이나 유지보수 비용 절감을 뜻하지는 않음

- 가상 공장 **48쌍** 조회 일치 및 백엔드 변경 실험 검토 범위 축소(`32 → 20 → 10`) 확인 (백엔드 검토 일지 · 가상 공장 대조 기록)

도입 판단 기준과 연구의 의의

> 지금 연구의 방향은 “온톨로지를 쓰면 LLM이 더 똑똑해진다”보다 “LLM이 사용하는 근거와 판단 조건을 명시하고 공유하며 변경과 오류를 추적할 수 있게 한다”입니다.

언제 Python과 타입 스키마로 충분한가?

> **서비스 하나 · 데이터 출처 하나 · 규칙 몇 개가 안정적으로 유지되는 상황**이라면 공유 Python 라이브러리와 타입 스키마로 끝내는 편이 더 단순하다.
> RDF·OWL·SHACL 도입에는 모델링·매핑·버전 관리라는 새로운 부담이 따른다.

도입 시점을 판단하는 네 가지 질문 (데이터 건수보다 중요)

1. 의미 중복	같은 업무 의미를 여러 팀·서비스가 반복해서 정의하고 있는가?
2. 변경 위치	자격·정책 규칙이 바뀔 때 수정할 위치를 찾기 어려운가?
3. 감사·설명	판단의 근거와 적용한 규칙 버전을 나중에 설명해야 하는가?
4. 저장소 전환	출처나 저장소(Neo4j · RDF · SQL)가 달라도 같은 해석을 유지해야 하는가?

연구의 핵심 방향과 검증 분리

> **지양하는 주장:** “온톨로지를 쓰면 LLM이 저절로 더 똑똑해진다.”

> **설득력 있는 방향:** “LLM이 사용하는 근거와 판단 조건을 명시하고 공유하며, 변경과 오류를 추적할 수 있게 한다.”

검증 축	무엇을 어떻게 보여줄 것인가
답변 품질 개선	다중 후보·조건 변경 등 특정 상황에서 구조화된 보고서가 가져오는 효과로 검증
구조 자체의 장점	규칙 재사용 · 변경 시 재검토 범위 축소 · 판단 근거 추적 가능성으로 별도 입증

- 답변 개선은 특정 상황에서 가져올 수 있는 효과로 검증하고 구조 자체의 장점은 **재사용·변경 절차·추적 가능성**으로 따로 평가